

Building and Securing a REST API for Mobile Money SMS Data

Software Engineering / REST API & DSA Project | **Team Data Wranglers**

Team Members: Precious Amarachi Azubuike, Aristote Henry Ngabo, Griphen Mweene

Repository: Momo_sms Project on GitHub

1. Introduction to API Security

Application Programming Interfaces (APIs) form the backbone of modern web architectures, enabling seamless communication between front-end clients, back-end servers, and third-party microservices. Because APIs expose database entities and backend functionality over public network protocols (HTTP/HTTPS), securing these endpoints is paramount to protecting data confidentiality, system integrity, and user privacy.

Without proper authentication and authorization mechanisms, API endpoints are vulnerable to unauthorized access, data exfiltration, parameter tampering, and denial-of-service attacks. In financial applications—such as Mobile Money (MoMo) SMS processing systems—securing API endpoints ensures that sensitive transaction records can only be queried, created, updated, or deleted by verified administrative clients.

In this project, we implemented Basic Authentication across all endpoints using HTTP headers. Basic Auth requires clients to transmit a credentials string composed of a Base64-encoded username and password (`admin:momo2026`) with every HTTP request. If credentials are missing or invalid, the API server rejects the request immediately with an HTTP 401 Unauthorized status code

2. Documentation of Endpoints & Visual Evidence

2.1 API Endpoint Reference

- **GET /transactions:** Retrieve all parsed SMS transactions | **Auth Required:** Yes
(`admin:momo2026`) | **Status:** 200 OK
- **GET /transactions/{id}:** Retrieve a specific transaction by ID | **Auth Required:** Yes
(`admin:momo2026`) | **Status:** 200 OK / 404 Not Found

- **POST /transactions:** Create a new transaction record | **Auth Required:** Yes
(admin:momo2026) | **Status:** 201 Created
- **PUT /transactions/{id}:** Update an existing transaction by ID | **Auth Required:** Yes
(admin:momo2026) | **Status:** 200 OK / 404 Not Found
- **DELETE /transactions/{id}:** Remove a transaction record by ID | **Auth Required:** Yes
(admin:momo2026) | **Status:** 200 OK / 404 Not Found

2.2 Test Results & Screenshots

Test Case 1: GET All Transactions (Authenticated)

- **Command:** `curl -i -u admin:momo2026 http://localhost:8000/transactions`
- **Result:** HTTP/1.0 200 OK returning the list of JSON transaction objects.
- **Evidence:**

```
4825696", "timestamp": "6 Jan 2025 8:13:05 PM", "raw_body": "You have received 74312 RWF from Linda Green (*****683) on your mobile money account at 2025-01-06 20:12:59. Message from sender: . Your new balance:103549 RWF. Financial Transaction Id: 30614825696.", {"id": 1646, "type": "transfer", "amount": 40000, "fee": 250, "balance": 63299, "sender": "me", "receiver": "Robert Brown", "phone": "25079077777", "transaction_ref": null, "timestamp": "6 Jan 2025 9:20:42 PM", "raw_body": ""*165*S*40000 RWF transferred to Robert Brown (25079077777) from 36521838 at 2025-01-06 21:20:35 . Fee was: 250 RWF. New balance: 63299 RWF. Kugura ama inite cg interineta kuri MoMo, Kanda *182*2*1# .*EN#"}, {"id": 1647, "type": "transfer", "amount": 15000, "fee": 250, "balance": 48049, "sender": "me", "receiver": "Samuel Carter", "phone": "25079077777", "transaction_ref": null, "timestamp": "6 Jan 2025 9:25:52 PM", "raw_body": ""*165*S*15000 RWF transferred to Samuel Carter (25079077777) from 36521838 at 2025-01-06 21:25:37 . Fee was: 250 RWF. New balance: 48049 RWF. Kugura ama inite cg interineta kuri MoMo, Kanda *182*2*1# .*EN#"}, {"id": 1648, "type": "transfer", "amount": 1500, "fee": 100, "balance": 46449, "sender": "me", "receiver": "Jane Smith", "phone": "25078899999", "transaction_ref": null, "timestamp": "6 Jan 2025 9:29:10 PM", "raw_body": ""*165*S*1500 RWF transferred to Jane Smith (25078899999) from 36521838 at 2025-01-06 21:29:03 . Fee was: 100 RWF. New balance: 46449 RWF. Kugura ama inite cg interineta kuri MoMo, Kanda *182*2*1# .*EN#"}, {"id": 1649, "type": "payment", "amount": 10000, "fee": 0, "balance": 36449, "sender": "me", "receiver": "Linda Green", "phone": null, "transaction_ref": "12490495576", "timestamp": "7 Jan 2025 12:00:48 AM", "raw_body": "TxId: 12490495576. Your payment of 10,000 RWF to Linda Green 27044 has been completed at 2025-01-07 00:00:41. Your new balance: 36,449 RWF. Fee was 0 RWF."}, {"id": 1650, "type": "transfer", "amount": 12000, "fee": 250, "balance": 24199, "sender": "me", "receiver": "Samuel Carter", "phone": "25078988888", "transaction_ref": null, "timestamp": "7 Jan 2025 3:13:59 PM", "raw_body": ""*165*S*12000 RWF transferred to Samuel Carter (25078988888) from 36521838 at 2025-01-07 15:13:52 . Fee was: 250 RWF. New balance: 24199 RWF. Kugura ama inite cg interineta kuri MoMo, Kanda *182*2*1# .*EN#"}, {"id": 1651, "type": "payment", "amount": 2500, "fee": 0, "balance": 21699, "sender": "me", "receiver": "Samuel Carter", "phone": null, "transaction_ref": "64907673038", "timestamp": "7 Jan 2025 6:45:08 PM", "raw_body": "TxId: 64907673038. Your payment of 2,500 RWF to Samuel Carter 11135 has been completed at 2025-01-07 18:45:01. Your new balance: 21,699 RWF. Fee was 0 RWF."}, {"id": 1652, "type": "payment", "amount": 200, "fee": 0, "balance": 21499, "sender": "me", "receiver": "Jane Smith", "phone": null, "transaction_ref": "32539170420", "timestamp": "7 Jan 2025 7:10:44 PM", "raw_body": "TxId: 32539170420. Your payment of 200 RWF to Jane Smith 71988 has been completed at 2025-01-07 19:10:37. Your new balance: 21,499 RWF. Fee was 0 RWF."}, {"id": 1653, "type": "transfer", "amount": 4000, "fee": 100, "balance": 17399, "sender": "me", "receiver": "Jane Smith", "phone": "25079077777", "transaction_ref": null, "timestamp": "7 Jan 2025 8:08:27 PM", "raw_body": ""*165*S*4000 RWF transferred to Jane Smith (25079077777) from 36521838 at 2025-01-07 20:08:20 . Fee was: 100 RWF. New balance: 17399 RWF. Kugura ama inite cg interineta kuri MoMo, Kanda *182*2*1# .*EN#"}, {"id": 1654, "type": "payment", "amount": 3500, "fee": 0, "balance": 13899, "sender": "me", "receiver": "Samuel Carter", "phone": null, "transaction_ref": "78103204977", "timestamp": "8 Jan 2025 9:15:00 AM", "raw_body": "TxId: 78103204977. Your payment of 3,500 RWF to Samuel Carter 91424 has been completed at 2025-01-08 09:14:53. Your new balance: 13,899 RWF. Fee was 0 RWF."}, {"id": 1655, "type": "payment", "amount": 4000, "fee": 0, "balance": 9899, "sender": "me", "receiver": "Robert Brown", "phone": null, "transaction_ref": "31387192219", "timestamp": "8 Jan 2025 1:01:26 PM", "raw_body": "TxId: 31387192219. Your payment of 4,000 RWF to Robert Brown 27045 has been completed at 2025-01-08 13:01:15. Your new balance: 9,899 RWF. Fee was 0 RWF."}, {"id": 1656, "type": "payment", "amount": 7000, "fee": 0, "balance": 2899, "sender": "me", "receiver": "Linda Green", "phone": null, "transaction_ref": "57476994247", "timestamp": "8 Jan 2025 1:37:21 PM", "raw_body": "TxId: 57476994247. Your payment of 7,000 RWF to Linda Green 29089 has been completed at 2025-01-08 13:37:13. Your new balance: 2,899 RWF. Fee was 0 RWF."}, {"id": 1657, "type": "bank deposit", "amount": 50000, "fee": null, "balance": 52899, "sender": "bank", "receiver": "me", "phone": null, "transaction_ref": null, "timestamp": "8 Jan 2025 10:10:34 PM", "raw_body": ""*113*R*A bank deposit of 50000 RWF has been added to your mobile m
```

Test Case 2: GET Transaction by ID (Authenticated)

- **Command:** `curl -i -u admin:momo2026 http://localhost:8000/transactions/1`
- **Result:** HTTP/1.0 200 OK returning transaction record ID 1.
- **Evidence:**

```
ent", "amount": 5000, "id": 1692}]
PS C:\Users\HP\OneDrive\Desktop\Momo_sms> curl.exe -i -u admin:momo2026 http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Sat, 26 Sep 2026 21:32:43 GMT
Content-Type: application/json
Content-Length: 426

{"id": 1, "type": "received_money", "amount": 9999, "fee": null, "balance": null, "sender": "Jane Smith", "receiver": "me", "phone": null, "transaction_ref": "76662021700", "timestamp": "10 May 2024 4:30:58 PM", "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
PS C:\Users\HP\OneDrive\Desktop\Momo_sms>
```

Test Case 3: Unauthorized Request Handling

- **Command:** `curl -i -u wrong:wrong http://localhost:8000/transactions`
- **Result:** HTTP/1.0 401 Unauthorized with WWW-Authenticate header. Access denied.
- **Evidence:**

```

C:\Users\HP\OneDrive\Desktop\Momo_sms> curl.exe -i -u wrong:wrong http://localhost:8000/transactions
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.14.3
Date: Sat, 26 Sep 2026 21:35:03 GMT
Content-Type: application/json
Content-Length: 41
WWW-Authenticate: Basic realm="MoMo SMS API"

{"error": "Invalid username or password"}
PS C:\Users\HP\OneDrive\Desktop\Momo_sms>

```

Test Case 4: POST Create Transaction

- **Command:** `curl.exe -i -X POST -u admin:momo2026 http://localhost:8000/transactions -H "Content-Type: application/json" -d '{"type": "payment", "amount": 5000, "sender": "me", "receiver": "Samuel Carter"}'`
- **Result:** HTTP/1.0 201 Created with newly assigned transaction ID 1693.
- **Evidence:**

```

C:\Users\HP\OneDrive\Desktop\Momo_sms> curl.exe -i -X POST -u admin:momo2026 http://localhost:8000/transactions -H "Content-Type: application/json" -d '{"type": "payment", "amount": 5000, "sender": "me", "receiver": "Samuel Carter"}'
HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.14.3
Date: Sat, 26 Sep 2026 21:41:58 GMT
Content-Type: application/json
Content-Length: 92

{"type": "payment", "amount": 5000, "sender": "me", "receiver": "Samuel Carter", "id": 1693}
C:\Users\HP\OneDrive\Desktop\Momo_sms>

```

Test Case 5: PUT Update Transaction

- **Command:** `curl.exe -i -X PUT -u admin:momo2026 http://localhost:8000/transactions/1 -H "Content-Type: application/json" -d '{"type": "received_money", "amount": 9999}'`
- **Result:** HTTP/1.0 200 OK returning updated fields ("amount": 9999).
- **Evidence:**

```

C:\Users\HP\OneDrive\Desktop\Momo_sms> curl.exe -i -X PUT -u admin:momo2026 http://localhost:8000/transactions/1 -H "Content-Type: application/json" -d '{"type": "received_money", "amount": 9999}'
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Sat, 26 Sep 2026 21:44:03 GMT
Content-Type: application/json
Content-Length: 426

{"id": 1, "type": "received money", "amount": 9999, "fee": null, "balance": null, "sender": "Jane Smith", "receiver": "me", "phone": null, "transaction_ref": "76662021700", "timestamp": "10 May 2024 4:30:58 PM", "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
C:\Users\HP\OneDrive\Desktop\Momo_sms>

```

Test Case 6: DELETE Transaction

- **Command:** `curl.exe -i -X DELETE -u admin:momo2026 http://localhost:8000/transactions/1`
- **Result:** HTTP/1.0 200 OK returning deletion confirmation message.
- **Evidence:**

```
C:\Users\HP\OneDrive\Desktop\Momo_sms>curl.exe -i -X DELETE -u admin:momo2026 http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.3
Date: Sat, 26 Sep 2026 21:44:27 GMT
Content-Type: application/json
Content-Length: 36

{"message": "Transaction 1 deleted"}
C:\Users\HP\OneDrive\Desktop\Momo_sms>
```

3. Results of Data Structures & Algorithms (DSA) Comparison

3.1 Empirical Benchmark Results

We benchmarked two search approaches over 1,600+ mobile money records:

- **Linear Search:** Sequential iteration through an array/list of transaction dictionaries.
- **Dictionary Lookup:** Direct key-value indexing using Python's native hash table implementation (id → transaction).

The timing measurements across target transactions produced the following performance metrics:

- **Dataset Size:** 1,691 transaction records
- **Iterations per Target ID:** 1,000 lookup operations
- **Average Linear Search Execution Time:** 0.00000106 seconds
- **Average Dictionary Lookup Execution Time:** 0.00000010 seconds
- **Average Speedup Factor:** ~10.85x faster using Dictionary Lookups

Evidence:

```
C:\Users\HP\OneDrive\Desktop\Momo_sms>python DSA/search_benchmark.py
Loaded 1691 records for benchmarking.
```

Target ID	Linear Search (s)	Dict Lookup (s)	Speedup Factor	
2	0.0000015	0.0000007	2.16	x
3	0.0000019	0.0000008	2.39	x
4	0.0000027	0.0000005	5.10	x
5	0.0000022	0.0000007	3.22	x
6	0.0000031	0.0000005	5.81	x
7	0.0000034	0.0000005	6.62	x
8	0.0000040	0.0000005	8.01	x
9	0.0000043	0.0000005	8.33	x
10	0.0000088	0.0000012	7.38	x
11	0.0000130	0.0000015	8.47	x
12	0.0000195	0.0000011	17.89	x
13	0.0000160	0.0000007	21.54	x
14	0.0000081	0.0000007	11.55	x
15	0.0000116	0.0000008	13.88	x
16	0.0000104	0.0000008	12.31	x
17	0.0000135	0.0000006	23.14	x
18	0.0000232	0.0000018	13.13	x
19	0.0000224	0.0000025	9.15	x
20	0.0000290	0.0000024	12.03	x
21	0.0000131	0.0000006	21.90	x
AVERAGE	0.0000106	0.0000010	10.85	x faster

3.2 Performance Analysis & Reflections

- **Why Dictionary Lookup Outperforms Linear Search:**
 - **Linear Search ($O(n)$ Time Complexity):** Scans elements sequentially starting from the head of the array until the target key is found or the end of the list is reached. In the worst-case scenario (or for non-existent IDs), the time required grows in direct linear proportion to the total dataset size n .
 - **Dictionary Lookup ($O(1)$ Time Complexity):** Employs a hash table mechanism. The target ID is processed through a cryptographic/numeric hash function to compute a direct memory index. Lookups execute in constant time $O(1)$ regardless of whether the dataset contains 10 elements or 1,000,000 elements.
- **Alternative Data Structures for Search Efficiency:**
 - **B-Trees / B+ Trees:** Ideal for disk-backed database indexing. B-Trees maintain balanced tree structures with logarithmic search time ($O(\log n)$) while enabling ultra-fast range queries (e.g., retrieving all transactions within a date range).
 - **Red-Black Trees / Balanced Binary Search Trees:** Maintains dynamically sorted elements with guaranteed $O(\log n)$ search, insertion, and deletion complexity.

4. Reflection on Basic Authentication Limitations & Stronger Alternatives

4.1 Vulnerabilities & Limitations of Basic Authentication

While Basic Auth is simple to implement using standard HTTP headers, it presents several major security drawbacks in production environments:

- **Unencrypted Base64 Encoding:** Basic Auth encodes credentials using Base64 (Authorization: Basic YWRtaW46bW9tbzlwMjY=). Base64 is an encoding scheme, not encryption. Anyone capturing network traffic via packet sniffing or man-in-the-middle (MitM) attacks can decode the header string instantly to reveal raw credentials.
- **Lack of Token Expiration / Revocation:** Basic Auth credentials remain static unless manually changed on the server. There are no built-in mechanisms to invalidate access automatically after a specific time-to-live (TTL).
- **Repeated Credential Transmission:** Because HTTP is stateless, the client must attach raw credentials to every single HTTP request, expanding the attack surface across network hops.
- **No Fine-Grained Role Controls:** Basic Auth provides binary access (either authenticated or unauthenticated) and does not natively support scoped user roles (e.g., distinguishing between Read-Only viewers vs. Full Administrative users).

4.2 Modern & Secure Alternatives

To address these vulnerabilities, modern API systems utilize state-of-the-art authentication frameworks:

- **JSON Web Tokens (JWT):**
 - **Mechanism:** Upon valid login, the server issues a digitally signed JWT containing payload claims (e.g., user ID, roles, expiration timestamp).
 - **Advantage:** The client attaches the JWT in the Authorization: Bearer token header. The back end validates the token statelessly using a public/private key pair without re-verifying password hashes or exposing raw credentials on every request. Tokens naturally expire via the exp claim.
- **OAuth 2.0 Framework:**
 - **Mechanism:** An industry-standard authorization framework that delegates identity verification to a dedicated Authorization Server (e.g., Auth0, Keycloak, or Google Identity).
 - **Advantage:** Third-party applications gain scoped access tokens (with granular permissions like read:transactions or write:transactions) without ever seeing or

storing user login credentials. It supports multi-factor authentication (MFA) and token revocation endpoints.